Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es la función principal del JDK (Java Development Kit)?

Seleccione una:

a. Permitir la edición de archivos JHTML y JCSS.
b. Servir como un servidor web para aplicaciones Java.
c. Proporcionar un entorno de ejecución para aplicaciones Java.
d. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java. ✓
e. Ninguna opción es correcta.

Justificación: El JDK aporta las herramientas para compilar (javac), depurar (jdb), empaquetar (jar) y ejecutar aplicaciones Java. La opción c, «entorno de ejecución», describe al JRE/JVM. Es el distractor más tentador porque el JDK también lo incluye, pero no es su función principal.

Fuente: Simulación APX V2 #21 · también aparece en: Java.pdf #105

La respuesta correcta es: Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.

Mi justificación
El JDK Ofrece las herramientas para que Java funcione, como el que crea el archivo bytecode (javac) y el empaquetador (jar)

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué herramienta se utiliza para empaquetar clases de Java en un archivo JAR (Java Archive)?

Seleccione una:

- a. jconsole
- b. java
- c. jps
- d. javac
- e. jar ✓

Justificación: La herramienta jar del JDK crea archivos .jar (por ejemplo, jar cf app.jar ...). javac compila, java ejecuta, jps lista los procesos de la JVM y jconsole los monitorea.

Fuente: Simulación APX V1 #6 · también aparece en: Java.pdf #37

La respuesta correcta es: jar

Mi justificación
El empaquetador es jar, es teoría básica

Pregunta 3

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes herramientas incluidas en el JDK se utiliza para monitorear y gestionar aplicaciones?

Seleccione una:

- a. VM ⊗
- b. javac
- c. jar
- d. jconsole
- e. java

Justificación: jconsole es la herramienta del JDK que se conecta por JMX a una JVM para monitorear memoria, hilos, clases y MBeans. java ejecuta aplicaciones pero no las monitorea, javac compila y jar empaqueta.

Fuente: Java.pdf #185

La respuesta correcta es: jconsole

Mi justificación
Honestamente no sabía la respuesta, fue al azar

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which declaration initializes a boolean variable?

Seleccione una:

- a. boolean k = 0;
- b. boolean m = null;
- c. boolean h = 1;
- d. boolean j = (1 < 5); ✓

Justificación: En Java, boolean solo acepta true, false o una expresión booleana, y (1 < 5) vale true. A diferencia de C, 0 y 1 no se convierten a boolean («int cannot be converted to boolean»), y null no se puede asignar a un tipo primitivo.

Fuente: Java DE #3 · también aparece en: Java D #3

La respuesta correcta es: boolean j = (1 < 5);

Mi justificación
Para inicializar una variable booleana necesitamos los valores true o false, o bien una comparación para que regrese uno de estos. XXX a) es inválida porque 0 es de tipo entero y no equivale a false. XXX b) es inválida porque null es para objetos y booleano es primitivo XXX c) es inválida porque 1 es de tipo entero y no equivale a true. VVV d) es válida porque (1 < 5) es una comparación que regresa false.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué declaraciones inicializan correctamente una variable booleana en Java?

Seleccione una o más de una:

- a. boolean d = (4 - 14);
- b. boolean f ="true";
- c. boolean b = (15 = 2);
- d. boolean c = (7 >= 7); ✓
- e. boolean e = (10 > 5 && 2 < 1); ✓
- f. boolean a = (3 < 6); ✓

Justificación: Solo compilan las expresiones de tipo boolean: 7 >= 7, (10 > 5 && 2 < 1) (vale false, pero es boolean) y 3 < 6. En cambio, 4 - 14 es int, "true" es String y (15 = 2) intenta asignar a un literal, así que ninguna de esas compila.

Fuente: Simulación APX V2 #30 · también aparece en: Cuestionario APX V3 #17, Java.pdf #124

Las respuestas correctas son: boolean c = (7 >= 7);, boolean e = (10 > 5 && 2 < 1);, boolean a = (3 < 6);

Mi justificación
Para inicializar una variable booleana necesitamos los valores true o false, o bien una comparación para que regrese uno de estos. XXX a) es una operación aritmética que regresa entero, b) es un String, c) no es una comparación porque debió usar ==. VVV d) e) y f) son comparaciones que regresan true o false

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es el tipo de resultado cuando se resta un valor short de un valor double en Java?

Seleccione una:

a. short
b. No se puede restar un short de un double.
c. int
d. float
e. double

Justificación: En una operación aritmética binaria, si un operando es double, el otro se promueve a double y el resultado es double. El distractor int solo aplica cuando ambos operandos son enteros de tipo int o menor.

Fuente: Simulación APX V1 #34 · también aparece en: Simulación APX V1 #36, Java.pdf #25, Java.pdf #60

La respuesta correcta es: double

Mi justificación
Cuando sumamos o restamos valores numéricos, el resultado es del tipo de mayor tamaño de los operandos. Como short < double, el resultado es de tipo double.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Main {
    public static void main(String[] args) {
        int a = 0;
        a++;
        System.out.println(a++);
        System.out.println(a);
    }
}
```

Seleccione una:

- a. 0 1
- b. 1 2 ✓
- c. 1 1
- d. 2 2

Justificación: Tras a++ la variable vale 1. println(a++) imprime el valor ANTES del post-incremento (1) y deja a en 2, y el último println imprime 2. La opción c (1 1) es el error típico de olvidar que el post-incremento sí modifica la variable después de usarla.

Fuente: Java E #4

La respuesta correcta es: 1 2

Mi justificación
El operador a++ se ejecuta después del print, así que se imprime a = 1, se suma 1 y luego se imprime a = 2

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Test{
    public static void main(String[] args){
        int b = 4;
        b--;
        System.out.print(--b);
        System.out.println(b);
    }
}
```

Seleccione una:

- a. 32
- b. 12
- c. 22 ✓
- d. 33

Justificación: b pasa de 4 a 3 con b--. Luego --b decrementa antes de usar el valor e imprime 2, y println(b) vuelve a imprimir 2, así que la salida es '22'. El distractor '32' confunde el predecremento con el posdecremento.

Fuente: Java DE #9 · también aparece en: Java Sesiones #32

La respuesta correcta es: 22

Mi justificación
El operador –b se ejecuta antes del print así que se obtiene b = 2, se imprime una vez b = 2 y se vuelve a imprimir b = 2.

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué imprime este código?

```
public static void main(String[] args) {
    boolean a = true;
    boolean b = false;
    boolean c = a && b || a;
    System.out.println(c);
}
```

Seleccione una:

- a. false
- b. Excepción
- c. true ✓
- d. null

Justificación: && tiene mayor precedencia que ||, así que se evalúa (a && b) || a = (true && false) || true = false || true = true. No puede haber excepción porque son primitivos boolean.

Fuente: Quiz APX #5

La respuesta correcta es: true

Mi justificación
El operador && tiene prioridad sobre || así que reemplazando: C = true && false || true C = false || true C = true

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
boolean x = true;
boolean y = false;
boolean result = x && y || !x;
System.out.println(result);
```

Seleccione una:

a. Runtime error

b. false ✓

c. true

d. Compilation error

Justificación: && tiene mayor precedencia que ||: (true && false) || !true = false || false = false. El distractor true surge al pensar que basta con que x sea true. Todos los operandos son boolean, así que no hay error de compilación ni de ejecución.

Fuente: Java.pdf #120 · también aparece en: Cuestionario APX V3 #29

La respuesta correcta es: false

Mi justificación
El operador && tiene prioridad sobre || así que reemplazando: C = true && false || !true C = false || false C = false

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Whats is the result?

```
public static void main(String[] args) {
    System.out.println("Result:" + 3 + 5);
    System.out.println("Result:" + (3 + 5));
}
```

Seleccione una:

a. Result: 8 Result: 8

b. Result: 35 Result: 8 ✓

c. Result: 8 Result: 35

d. Result: 35 Result: 35

Justificación: En la primera línea, "Result:" + 3 ya es String, así que el 5 se concatena y queda "Result:35". En la segunda, los paréntesis obligan a sumar 3 + 5 primero, y queda "Result:8".

Fuente: Java Sesiones #38

La respuesta correcta es: Result: 35 Result: 8

Mi justificación
Cuando ejecutamos print, al aparecer un String, todos los + siguientes se convierten en concatenación a menos que estén encerrados en una operación aritmética entre paréntesis. Entonces: El primer print imprime Result: 35 El segundo print respeta la operación 3+5 = 8 e imprime Result: 8

Pregunta 12

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public static void main(String[] args) {
    System.out.println("Result: " + 2 + 3 + 5);
    System.out.println("Result: " + 2 + 3 * 5);
}
```

Seleccione una:

- a. Result: 10 Result: 30
- b. Result: 25 Result: 10
- c. Result: 235 Result: 215 ✓
- d. Result: 215 Result: 215
- e. Compilation fails.

Justificación: La concatenación se evalúa de izquierda a derecha: "Result: " + 2 da un String, luego se concatenan 3 y 5 y queda "Result: 235". En la segunda línea * tiene mayor precedencia, así que 3*5 = 15 se calcula primero y queda "Result: 215". El distractor d olvida que en la primera línea no hay multiplicación.

Fuente: Java Sesiones #2

La respuesta correcta es: Result: 235 Result: 215

Mi justificación
Cuando ejecutamos print, al aparecer un String, todos los + siguientes se convierten en concatenación a menos que estén encerrados en una operación aritmética entre paréntesis. Esto no aplica para *, que siempre se respetará la multiplicación Entonces: El primer print imprime Result: 235 El segundo print respeta la operación 3*5 = 15 e imprime Result: 215

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es el resultado de la operación 9 | 4 en Java?

```
System.out.println(9 | 4);
```

Respuesta: 13

Justificación: Entre enteros, | es el OR bit a bit: 9 = 1001 y 4 = 0100, así que el OR da 1101 = 13. El error típico es confundirlo con el OR lógico (||), que no se aplica a int, o sumar mal los bits.

Fuente: Java.pdf #75

La respuesta correcta es: 13

Mi justificación
La operación 9|4 utiliza OR binario así que se descompone cada número a binario y se hace el OR con cada dígito: 9 = 1001 4 = 0100 R = 1101 = 13

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles de las siguientes son formas válidas de inicializar una variable booleana en Java?

Seleccione una o más de una:

- [x] a. boolean flag = Boolean.FALSE;
- [x] b. boolean flag = true;
- [ ] c. boolean flag = 0;
- [x] d. boolean flag = false;
- [ ] e. boolean flag = "false";

Justificación: boolean solo acepta true/false o una expresión booleana. Boolean.FALSE se desempaqueta automáticamente (unboxing) a false. 0 no es booleano en Java (a diferencia de C) y "false" es un String, así que ninguno de esos dos compila.

Fuente: Simulación APX V2 #8 · también aparece en: Java.pdf #76

Las respuestas correctas son: boolean flag = Boolean.FALSE;, boolean flag = true;, boolean flag = false;

Mi justificación
XXX c) es inválido porque 0 es entero, e) es inválido porque es String. VVV a) es válido porque uno de los valores discretos que permite Boolean es false, b) es true el cual es booleano y d) es false el cual es booleano.

Pregunta 15

Parcialmente correcta

Se puntúa 0,75 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles de los siguientes operadores son válidos para variables booleanas en Java?

Seleccione una o más de una:

- a. && ✓
- b. |
- c. +
- d. || ✓
- e. == ✓

Ha seleccionado correctamente 3.

Justificación: Con boolean son válidos los lógicos de cortocircuito && y ||, el lógico sin cortocircuito | (también & y ^) y la igualdad ==. El operador + no está definido para boolean.

Fuente: Simulación APX V1 #44 · también aparece en: Java.pdf #39

Las respuestas correctas son: &&, |, ||, ==

Mi justificación
Los operadores válidos para booleanos son los de comparación lógica, en este caso son &&, || y ==.

Pregunta 16

Parcialmente correcta

Se puntúa 0,50 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which two declarations will compile?

```
14.    public static void main(String[] args) {
15.        Int a, b, c = 0;
16.        int a, b, c;
17.        int g, int h, int i = 0;
18.        int d, e, f;
19.        Int k, l, m, = 0;
20.    }
```

Seleccione una o más de una:

- a. Line 15. ✗
- b. Line 16. ✓
- c. Line 17.
- d. Line 18. ✓
- e. Line 19.
- f. Line 20.

Mi justificación
XXX c) es inválido porque solo debes poner una vez int si usas comas, e) es inválido porque es Integer y no Int, f) es una llave nada más. VVV a) b) y c) respetan las reglas que rompieron c) y d)

Pregunta 17

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Cuando se asigna una variable byte (no final) a un carácter (char) en Java sin cast explícito, ¿cuál es el tipo de resultado?

Seleccione una:

- a. char ⊗
- b. int
- c. byte
- d. short
- e. No puedes asignar un byte a un carácter.

Justificación: La conversión de byte a char es de ampliación y estrechamiento a la vez (JLS 5.1.4): byte tiene signo y char no, así que asignar una variable byte a un char sin cast no compila («possible lossy conversion from byte to char»). «char» sería el tipo solo si se escribe (char) b, y no hay promoción a int porque no hay operación aritmética. Matiz: si el byte es una constante de compilación con un valor representable (final byte b = 65; char c = b;), el compilador sí la acepta. Por eso el enunciado dice «variable byte (no final)».

Nota sobre el material original: El material original no traía clave para esta pregunta y su redacción era ambigua: con cast explícito, o con una constante final, sí se puede asignar y el resultado es char. Se añadió «variable byte (no final)» y «(char) sin cast explícito» al enunciado para que tenga una sola respuesta.

Fuente: Simulación APX V2 #25 · también aparece en: Simulación APX V4 #40, Java.pdf #113

La respuesta correcta es: No puedes asignar un byte a un carácter.

?

Mi justificación
Como char es más grande que byte, el resultado es char.

Pregunta 18

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is printed out when the program is executed?

```
public class MainMethod{
    void main(){
        System.out.println("one");
    }
    static void main(String args){
        System.out.println("two");
    }
    public static final void main(String[] args){
        System.out.println("three");
    }
    void mina(Object[] args){
        System.out.println("four");
    }
}
```

Seleccione una:

a. There is no output
b. three ✓
c. two
d. four
e. one

Justificación: La JVM usa como punto de entrada 'public static void main(String[] args)'. El modificador final es legal y no impide que sea el main. Las demás versiones (main(), main(String), mina(Object[])) son simples métodos que nunca se invocan. El distractor 'two' confunde main(String) con main(String[]).

Fuente: Java DE #1 · también aparece en: Java Sesiones #6

La respuesta correcta es: three

Mi justificación
El método main es public static void main con final opcional. Así que la única que se ejecuta es la que imprime “three”.

Pregunta 19

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which two will compile, and can be run successfully using the following command?
java Fred1 hello walls

Seleccione una o más de una:

a. abstract class Fred1 { public static void main(String[] args) { System.out.println(args[2]); } }
b. class Fred1 { public static void main(String args) { System.out.println(args[1]); } }
c. class Fred1 { public static void main(String[] args) { System.out.println(args); } }
d. class Fred1 { public static void main(String[] args) { System.out.println(args[1]); } }

Justificación: Con 'java Fred1 hello walls', args tiene 2 elementos. La c imprime la referencia del arreglo (algo como [Ljava.lang.String;@...) y la d imprime args[1] = walls: ambas compilan y terminan bien. La a compila (una clase abstracta puede tener main), pero args[2] lanza ArrayIndexOutOfBoundsException. La b no compila: args es un String, no un arreglo, así que args[1] da «array required, but String found»; además main(String) no sería un punto de entrada válido.

Fuente: Java DE #5 · también aparece en: Java D #4

Las respuestas correctas son: class Fred1 { public static void main(String[] args) { System.out.println(args); } }, class Fred1 { public static void main(String[] args) { System.out.println(args[1]); } }

Mi justificación
XXX a) no compila porque hello walls tiene 2 argumentos, ósea que el array args[] llega hasta 1, entonces args[2] causa un error, b) no compila porque args debe ser un arreglo siempre. VVV c) y d) cumplen adecuadamente.

Pregunta 20

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the narrowest valid return type for the calculate Method?

```
public class Test {
    ___ doMath (long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. float
- b. short
- c. int
- d. long ✓
- e. double
- f. byte

Justificación: l * 10 es long, y s / long también es long, así que el retorno debe admitir un long. long, float y double compilan por ampliación, y el más estrecho de ellos es long. int, short y byte fallan por conversión con pérdida.

Fuente: Java.pdf #145

La respuesta correcta es: long

Mi justificación
Una división de short entre long da como valor mínimo long porque short < long siempre y cuando el resultado sea exacto.

Pregunta 21

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the narrowest valid return type for the doMath method?

```
public class Test {
    ___ doMath (long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. float
- b. short
- c. int
- d. long ✓
- e. double
- f. byte

Justificación: s / (l * 10) es de tipo long, porque l es long y promueve toda la expresión. El retorno acepta long o tipos a los que long se amplía implícitamente (float, double), mientras que int, short y byte fallan con «possible lossy conversion from long to ...». De los tipos válidos, el más estrecho es long.

Nota sobre el material original: En el formulario original la pregunta no tenía enunciado (solo el código); se añadió el de la versión del PDF de Java (pregunta 145), «narrowest valid return type», que es la que da una sola respuesta.

Fuente: Cuestionario APX V3 #19 · también aparece en: Simulación APX V4 #18

La respuesta correcta es: long

Mi justificación
Una división de short entre long da como valor mínimo long porque short < long siempre y cuando el resultado sea exacto.

Pregunta 22

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Dado el siguiente código ¿Cuál es el tipo de retorno válido punto flotante más restringido para el método de cálculo?

```
public class Test {
    _____ doMath(long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. long ⊗
- b. float
- c. short
- d. int
- e. byte
- f. double

Justificación: s / (l * 10) es de tipo long por promoción numérica. Un long se amplía implícitamente a float o a double, así que ambos compilan, y float (32 bits) es el tipo de punto flotante más restringido. double también compila pero es más ancho; long compila pero no es de punto flotante; int, short y byte dan error de conversión con pérdida.

Fuente: Java.pdf #52 · también aparece en: Simulación APX V1 #9

La respuesta correcta es: float

Mi justificación
Una división de short entre long da como valor mínimo long porque short < long siempre y cuando el resultado sea exacto.

Pregunta 23

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Which of the following lines of code correctly defines the main method in a Java program?

Seleccione una o más de una:

- a. static void main(String args[]) {}
- b. public static int main(String[] args) {}
- c. public static void main(String... String) {}
- d. public static void main(String[] args) {}
- e. private static void main(String[] args) {}

Justificación: El punto de entrada clásico debe ser public static void y recibir un String[] o varargs String.... El nombre del parámetro puede ser cualquier identificador válido, incluso 'String'. Por eso c y d son válidas. La a (sin public) y la e (private) compilan, pero el lanzador no las acepta como main, y la b devuelve int.

Fuente: Java.pdf #201

Las respuestas correctas son: public static void main(String... String) {}, public static void main(String[] args) {}

Mi justificación
El método main es public static void main con final opcional y args como un arreglo. Así que las únicas que cumplen son a) y d).

Pregunta 24

Parcialmente correcta

Se puntúa 0,33 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Which of the following statements about the java -jar command, which is used to execute Java programs packaged in JAR (Java Archive) files, are true?

Seleccione una o más de una:

- a. The java -jar command can dynamically add libraries to the classpath using the -cp or -classpath options. ✗
- b. The java -jar command ignores the CLASSPATH environment variable and command-line classpath settings.
- c. The java -jar command can execute a JAR file without specifying the main class if the JAR file contains multiple main classes.
- d. The java -jar command requires the JAR file to have a META-INF/MANIFEST.MF file specifying the main class. ✓
- e. The java -jar command can only be used to execute JAR files that are executable JARs, i.e., those that contain the META-INF/MANIFEST.MF file with the Main-Class attribute. ✓

Ha seleccionado correctamente 2.

Justificación: Con java -jar, el classpath lo define el propio JAR (y el Class-Path del manifiesto), así que se ignoran -cp, -classpath y la variable CLASSPATH. Además, el manifiesto debe declarar Main-Class, por lo que solo corren los JAR ejecutables. La opción a es la trampa habitual: -cp no tiene efecto junto a -jar.

Fuente: Java.pdf #191

Las respuestas correctas son: The java -jar command ignores the CLASSPATH environment variable and command-line classpath settings., The java -jar command requires the JAR file to have a META-INF/MANIFEST.MF file specifying the main class., The java -jar command can only be used to execute JAR files that are executable JARs, i.e., those that contain the META-INF/MANIFEST.MF file with the Main-Class attribute.

Mi justificación
La respondí al azar.

RESULTADO = 17/24 = 7.1